Slide 1 · 0:00–0:40

Timoni + CUE, alongside Flux

Start with the tools' responsibilities. CUE describes data and constraints; Timoni uses it to build and package Kubernetes objects. Kustomize transforms existing objects through bases, patches and other operations. Flux continuously reconciles declared state in the cluster. These responsibilities can be combined in a delivery pipeline. Our question is what an explicit configuration contract adds. We will use one replica count to explain constraints and composition, then apply the idea to a small kro API.

Timing: 40 seconds. Planned total: 10:00.

Sources: Timoni · Kustomize · Flux

Slide 2 · 0:40–1:20

CUE’s creator: Marcel van Lohuizen

Marcel van Lohuizen created CUE. At Google he was on the founding Borg team, where he created the Borg Configuration Language and core tooling. BCL was later generalized into GCL. CUE is a different language informed by that experience. Its design returns to constraint composition and graph unification. The useful connection for us: configuration can carry allowed values and reusable structure in the same language. The next two slides explain that model with small examples.

Timing: 40 seconds. Planned total: 10:00.

Sources: Marcel’s biography · CUE history

Slide 3 · 1:20–2:40

Types and values share one model

Think of an integer type as the set of all integers, a range as a smaller set, and a concrete value as a singleton. CUE represents them within the same model. Read the expression from inside the parentheses first. The value must be an integer, at least one and at most ten. Ampersand means that every constraint must hold. The vertical bar offers alternatives, and the star marks two as the preferred default.

Walk through the three cases. With no supplied replica value, evaluation selects two. Three satisfies the range and becomes the concrete result. Zero satisfies neither alternative, so evaluation fails. A string such as “three” also fails. The default is a preference, so it does not prevent valid input from selecting three. This expression comes from the included Deployment module. Exporting resources requires concrete values; a range alone does not choose a replica count. Pause on the cases before moving on.

Timing: 80 seconds. Planned total: 10:00.

Sources: Constraints · Defaults

Slide 4 · 2:40–3:50

Composition means satisfying both

The hash introduces a reusable CUE definition. Here Config contains a replica constraint without a default, so supplying a concrete value makes the narrowing operation clear. Combining Config with replicas three gives the structure shown on the right. Reversing those two operands gives the same result. CUE calls this operation unification.

The computer-science connection is intersection: the result must satisfy both sources of information. Unification is commutative, associative and idempotent. Adding the same compatible constraint twice does not change the answer. Supplying zero conflicts with this range; combining replicas three with replicas four conflicts too. That is different from an override operation in which one concrete value replaces another. Defaults support valid choices, as on the previous slide. These algebraic properties describe CUE unification; they do not imply that patches, command-line processing or an entire delivery pipeline can run in arbitrary order.

Timing: 70 seconds. Planned total: 10:00.

Sources: Unification · Definitions · Composition properties

Slide 5 · 3:50–5:00

Transform YAML or constrain inputs

Both examples select three replicas. Kustomize starts from a Deployment with two replicas and transforms that field to three. The replicas transformer is an alternative to a patch for this field. On the right, CUE combines an input with a definition that includes a default and a permitted range. The Timoni module connects values.replicas to the Deployment's replica count.

Now change the input to zero. Kustomize can render zero, which is a valid Kubernetes replica count. It does not encode our example's minimum of one. CUE rejects zero because that minimum is part of the module's contract. This shows where the rule is expressed: schema tools, policies and CI checks can also accompany Kustomize. Both approaches reuse configuration. CUE makes reusable data and constraints part of one language; Kustomize offers familiar object transformations. Use the matching offline examples to inspect both.

Timing: 70 seconds. Planned total: 10:00.

Sources: Kustomize operations · Offline comparison

Slide 6 · 5:00–6:10

Timoni packages CUE modules

Timoni provides a Kubernetes workflow around CUE. A module exposes a configuration interface and connects those inputs to resources. Authors can vendor Kubernetes API or custom-resource schemas so the resource definitions are checked as well as their own configuration rules. This is explicit schema integration: a CUE file alone does not automatically know every Kubernetes API.

The first command renders our local WebApp RGD module. The second runs module validation against its included schema. These local commands do not contact Kubernetes. Timoni can package CUE sources and included schemas as a versioned OCI artifact; consumers build it with their own values. Its CLI also offers apply and instance inventory. Continuous reconciliation remains a separate responsibility. The small Deployment example omits vendored Kubernetes schemas; the RGD example includes the upstream kro schema. Structural validation is useful evidence, but controller behavior still needs runtime testing.

Timing: 70 seconds. Planned total: 10:00.

Sources: Timoni concepts · Modules · CRD schemas

Slide 7 · 6:10–7:30

Use shared CUE structure for a kro API

kro turns a ResourceGraphDefinition into a Kubernetes API and reconciles each instance's resources. This example is a namespaced WebApp backed by one Deployment. CUE combines shared conventions and the vendored kro 0.9.4 schema, then adds graph-specific fields. Both panels are excerpts; the full module includes the remaining Deployment fields.

Read the right panel from schema to resource. Default two and the range are part of the generated API. A WebApp can supply replicas three; the CEL expression reads three and kro is expected to reconcile three replicas. Changing the API default does not rewrite existing instances.

Timoni preserves the expression as data; kro evaluates it later. The RGD is cluster-scoped; WebApps and their Deployments are namespaced. Runtime behavior remains illustrative. When delivering RGDs through Flux, preserve CEL expressions during Flux substitution; the handoff example explains how.

Timing: 80 seconds. Planned total: 10:00.

Sources: kro v0.9.4 · Complete module, including vendored schema · Flux values handoff

Slide 8 · 7:30–9:00

Where Bicep values meet Kubernetes

Start with the team’s handoff: Bicep or CI publishes ConfigMaps and Secrets; Flux’s substituteFrom reads their values after Kustomize builds. The code outlines a separate consumer validation step.

A Bundle groups module instances. Runtime maps ConfigMap replicas to DEMO_REPLICAS and Secret token to DEMO_TOKEN. Attributes parse a number and a string into hidden fields. The consuming values then unify these inputs with an integer range, one to ten, and a nonempty-string contract. Keep the constraints at the consuming fields, separate from injection. The complete example factors them into `#InfraValues`; Bundle metadata, module reference and namespace are omitted here.

Trace one input: ConfigMap string three becomes integer three. Zero, a fraction, nonnumeric input or an empty or missing token fails validation. CUE needs concrete values: checking a placeholder cannot validate a later Flux substitution. Bicep’s typed outputs do not carry this consumer contract.

Timoni Runtime is an explicitly invoked CLI step. Flux does not run it automatically; input changes need to trigger it again. Keeping Flux and validating producer outputs before publishing is another option. The linked example checks synthetic inputs offline.

Timing: 90 seconds. Planned total: 10:00.

Sources: Flux substitution · Timoni Runtime · Offline handoff example

Slide 9 · 9:00–10:00

Choose the abstraction you need

Close on the tradeoff. CUE lets configuration authors express reusable structure and contracts together. Unification combines compatible rules and exposes contradictions during evaluation. Timoni turns that language into modules, schema integration and a Kubernetes packaging workflow. These benefits come with a language to learn, diagnostics to understand, and schemas and toolchains to maintain.

Kustomize remains a good fit for straightforward transformations, especially when existing validation already covers the required policies. Flux can continue reading infrastructure values and reconciling resources. CUE is most interesting when several sources of shared structure and rules must compose reliably. The README links offline output comparisons, the runtime-input contract and additional study material. Finish at ten minutes and invite questions afterward.

Timing: 60 seconds. Planned total: 10:00.